

Basic Node.js

Full-Stack Web Development

EGCI427 Web Programming — Lecture 06

01

What is Node.js?

Architecture, V8 engine, use cases

02

Blocking vs Non-Blocking I/O

Event loop & async patterns

03

Node Package Manager (npm)

Installing & managing packages

04

Core Modules

fs, path, os, http, url, events

05

Custom Modules

exports / require pattern

06

HTTP Server

Serving HTML and JSON

07

Express.js

Routing, middleware, REST APIs

08

File System Deep Dive

Read, write, stream

09

Async Patterns

Callbacks, Promises, async/await

10

Building a Mini REST API

Full worked example + practice

01

What is Node.js?

Runtime environment, V8 engine, and why it matters

What is Node.js?

Open-source, cross-platform JavaScript runtime built on Chrome's V8 engine.



JavaScript Everywhere

Run JS on the server —
same language as the browser



Event-Driven Architecture

Non-blocking I/O handles
thousands of connections



Rich Ecosystem

npm has 2M+ packages —
largest registry in the world



High Performance

V8 compiles JS to native
machine code at runtime

Who uses Node.js?

Netflix

LinkedIn

Uber

NASA

PayPal

Walmart

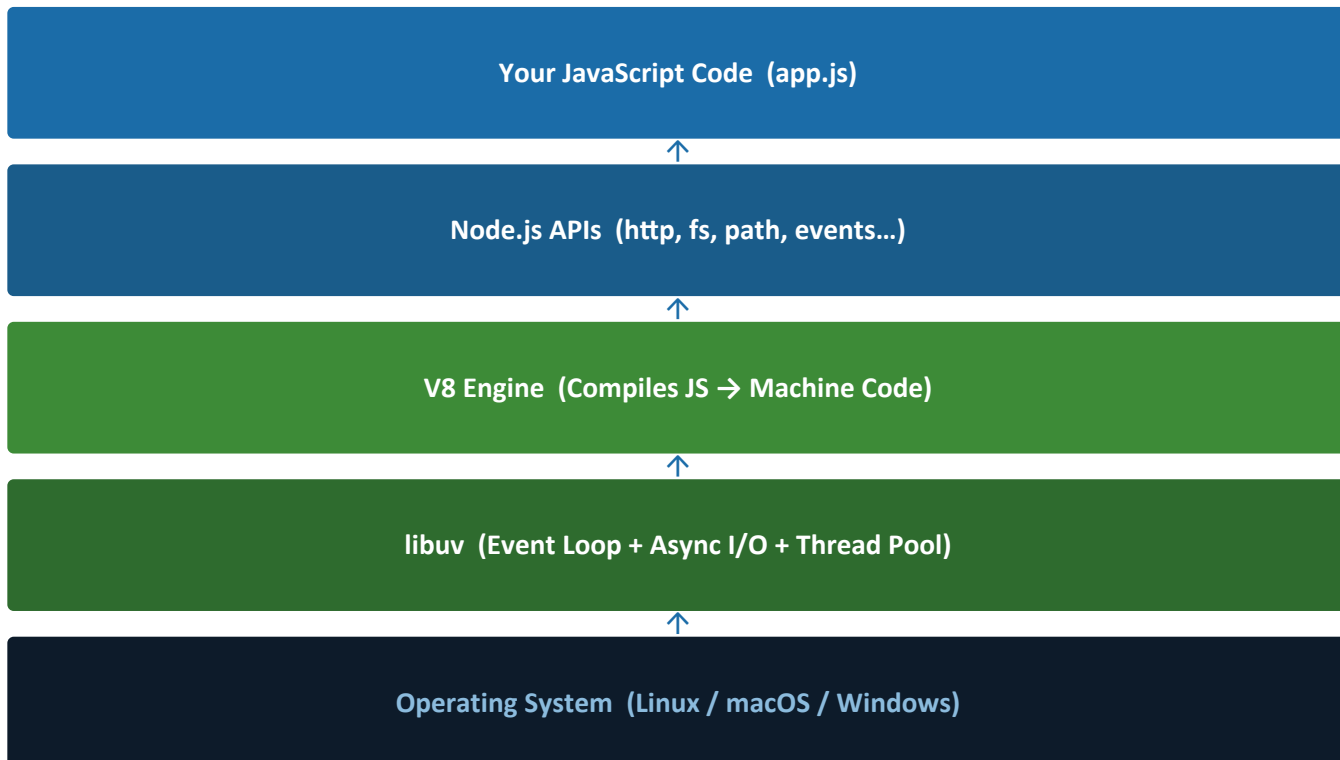
Microsoft

Groupon



Key insight: Node.js is single-threaded but handles concurrency through its event loop — perfect for I/O-heavy applications like web APIs and real-time apps.

Understanding the engine under the hood



V8: Developed by Google, used in Chrome. Compiles JS to native machine code (x86, ARM, etc.) — not interpreted. libuv: C library providing non-blocking I/O using OS kernel features (epoll, kqueue).

02

Blocking vs Non-Blocking I/O

The core concept that makes Node.js fast

Blocking: wait for each operation to finish. Non-Blocking: register a callback, move on immediately.

❌ Blocking (Synchronous) — blocks the thread

```
// BLOCKING — synchronous (waits at each step)
const fs = require('fs');

console.log('Start');

// This BLOCKS the entire program until file is read
const data = fs.readFileSync('./data.txt', 'utf8');
console.log('File content:', data);    // runs after file is read

const data2 = fs.readFileSync('./data2.txt', 'utf8');
console.log('File 2:', data2);        // runs after file 2 is read

console.log('End');
// Output: Start → File content → File 2 → End (sequential)
```

✅ Non-Blocking (Async) — keeps thread free

```
// NON-BLOCKING — asynchronous (callback-based)
const fs = require('fs');

console.log('Start');

// Registers callback, does NOT block
fs.readFile('./data.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('File content:', data); // runs later
});

fs.readFile('./data2.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('File 2:', data);       // may run before File 1!
});

console.log('End');
// Output: Start → End → File content → File 2 (order not guaranteed)
```

Why Non-Blocking wins for web servers:

Blocking Server

Req 1
(blocked)

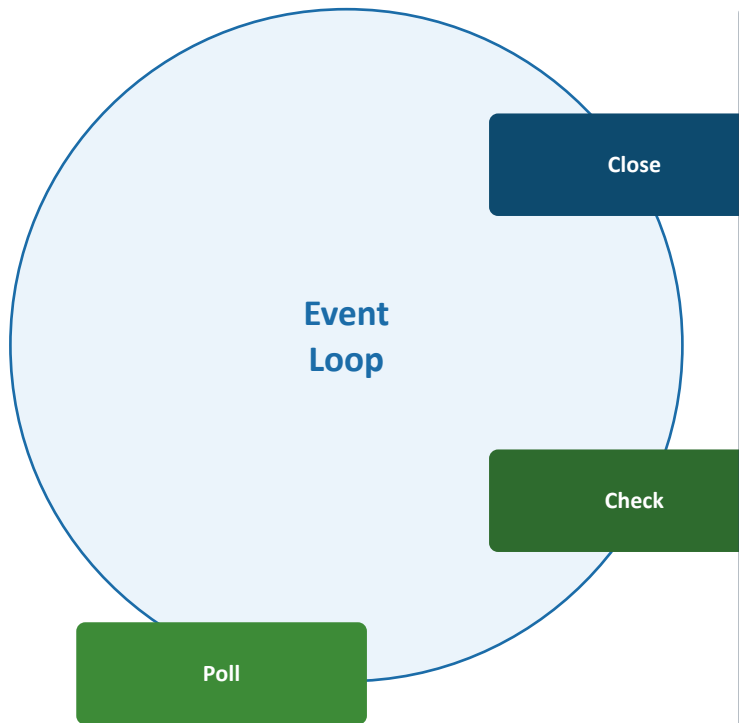
Req 2
waits

Req 3
waits

Node.js Server

All 3 requests served concurrently — CPU free while waiting for I/O

The secret to Node's concurrency — a single thread that never blocks



Event Loop Order Demo

```
// Understanding the event loop
console.log('1: Start');           // synchronous

setTimeout(() => {
  console.log('3: setTimeout');    // Timer phase
}, 0);

Promise.resolve().then(() => {
  console.log('2b: Microtask');    // Microtask queue (after
  // current task)
});

setImmediate(() => {
  console.log('4: setImmediate');  // Check phase
});

console.log('2: End of script');    // synchronous

// Output order:
// 1: Start
// 2: End of script
// 2b: Microtask ← Promises run before next event loop tick
// 3: setTimeout
// 4: setImmediate
```

03

Node Package Manager (npm)

Install, manage, and publish JavaScript packages

npm ships with Node.js. It's the world's largest software registry with 2M+ packages.

Essential npm Commands

```
# Initialize a new project
npm init -y                # creates package.json with
defaults

# Install packages
npm install express        # install + save to
dependencies
npm install nodemon --save-dev # install to devDependencies
only
npm install                # install all deps from
package.json

# Uninstall
npm uninstall express

# Update packages
npm update express
npm outdated              # check for newer versions

# Run scripts defined in package.json
npm start                 # runs "scripts.start"
npm run dev               # runs "scripts.dev"
npm test                  # runs "scripts.test"

# View package info
npm info express
npm list --depth=0        # list installed packages
```

package.json Explained

```
// package.json — project manifest
{
  "name": "my-node-app",
  "version": "1.0.0",
  "description": "EGCI427 Node.js App",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "node test.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}

// Version notation:
// "^4.18.2" → allow minor/patch updates (4.x.x)
// "~4.18.2" → allow patch updates only (4.18.x)
// "4.18.2" → exact version only
```

04

Core Modules

Built-in modules — no npm install needed

Built-in Core Modules — No Installation Required

04 · Core Modules

Require them directly. These are part of Node.js itself.

`fs`

Read/write files, directories, streams

`path`

Work with file & directory paths

`http/https`

Create HTTP servers and make requests

`os`

Operating system info & utilities

`url`

Parse and format URLs

`events`

EventEmitter — pub/sub pattern

`crypto`

Hashing, encryption, secure random

`stream`

Readable, Writable, Transform streams

`buffer`

Binary data handling

`process`

Current process info, env vars, exit

`readline`

Read input line-by-line (stdin)

`child_process`

Spawn OS commands and processes

```
const fs = require('fs') | const path = require('path') | const http = require('http')
```

Always use path to build file paths — never string concatenation (breaks on Windows vs Unix)

path Module Examples

```
const path = require('path');

// Join path segments safely (handles / vs \ on OS)
const filePath = path.join(__dirname, 'data', 'users.json');
console.log(filePath);
// /home/user/project/data/users.json (Linux/Mac)
// C:\Users\project\data\users.json (Windows)

// Get just the filename
path.basename('/home/user/file.txt'); // 'file.txt'
path.basename('/home/user/file.txt', '.txt'); // 'file'

// Get just the directory
path.dirname('/home/user/file.txt'); // '/home/user'

// Get file extension
path.extname('/home/user/file.txt'); // '.txt'

// Parse path into parts
const parsed = path.parse('/home/user/file.txt');
// { root: '/', dir: '/home/user', base: 'file.txt',
//   ext: '.txt', name: 'file' }

// Build absolute path
path.resolve('data', 'users.json');
// → /current/working/directory/data/users.json

// Check if path is absolute
path.isAbsolute('/home/user'); // true
path.isAbsolute('./data'); // false
```

Why __dirname?

- ▶ `__dirname` = absolute path to current file's directory
- ▶ `__filename` = absolute path to current file
- ▶ Always use `__dirname + path.join()` for file paths
- ▶ Never rely on `process.cwd()` for module paths

Real-World Pattern

```
// Common pattern in Node.js projects
const path = require('path');

// Load a JSON file relative to this script
const usersFile = path.join(__dirname, 'data',
  'users.json');

// Build a public folder path
const publicDir = path.join(__dirname, '..',
  'public');

// In Express — serve static files:
// app.use(express.static(path.join(__dirname,
  'public')));
```

Useful for system-aware applications and deployment scripts

os Module — System Information

```
const os = require('os');

// Platform and architecture
console.log(os.platform()); // 'linux', 'darwin', 'win32'
console.log(os.arch());     // 'x64', 'arm64', 'ia32'
console.log(os.release());  // '5.15.0-91-generic' (kernel)
console.log(os.type());     // 'Linux', 'Darwin', 'Windows_NT'

// CPU information
console.log(os.cpus().length); // number of CPU cores
console.log(os.cpus()[0].model); // 'Intel(R) Core(TM) i7...'
console.log(os.loadavg());     // [1.2, 1.5, 1.1] (1, 5, 15 min avg)

// Memory
console.log(os.totalmem()); // total RAM in bytes
console.log(os.freemem());  // free RAM in bytes
const usedPercent = ((1 - os.freemem()/os.totalmem())*100).toFixed(1);
console.log(`Memory used: ${usedPercent}%`);

// Network interfaces
const nets = os.networkInterfaces();
console.log(nets); // all network interfaces with IP addresses

// Hostname and home directory
console.log(os.hostname()); // 'my-server-01'
console.log(os.homedir());  // '/home/user'
console.log(os.tmpdir());   // '/tmp'

// End of line (newline character for this OS)
console.log(os.EOL === '\n' ? 'Unix' : 'Windows');
```

Real-World Usage

```
// Practical use: adaptive server config
const os = require('os');

const config = {
  maxWorkers: os.cpus().length, // match CPU
  count
  tmpPath:    os.tmpdir(),       // OS temp
  folder
  hostname:   os.hostname(),     // server
  identity
};

console.log(`Starting ${config.maxWorkers} workers on
${config.hostname}`);

// Memory health check endpoint
function getMemoryStatus() {
  const total = os.totalmem();
  const free  = os.freemem();
  const used  = total - free;
  return {
    totalMB: (total / 1024 / 1024).toFixed(0),
    usedMB:  (used / 1024 / 1024).toFixed(0),
    freeMB:  (free / 1024 / 1024).toFixed(0),
    percent: ((used / total) * 100).toFixed(1),
  };
}
```

05

Custom Modules

Organize code with `require()` and `module.exports`

Every .js file in Node.js is a module. Use `module.exports` to share functions, classes, and values.

mathUtils.js — The Module

```
// mathUtils.js — defining the module

// Named exports (attach to module.exports object)
const PI = Math.PI;

function add(a, b) { return a + b; }
function subtract(a, b) { return a - b; }
function multiply(a, b) { return a * b; }
function divide(a, b) {
  if (b === 0) throw new Error('Division by zero!');
  return a / b;
}
function circleArea(r) { return PI * r * r; }

// Export what you want other files to use
module.exports = { PI, add, subtract, multiply, divide,
  circleArea };

// _____
// Alternative: ES6 shorthand (same result)
// exports.add = (a, b) => a + b; ← old style, avoid
```

Exporting a Class

```
// Exporting a class from a module
// user.js
class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
    this.id = Date.now();
  }
}
```

app.js — Using the Module

```
// app.js — using the module

// Require with ./ means "relative to this file"
const math = require('./mathUtils');

console.log(math.add(5, 3));           // 8
console.log(math.circleArea(5));       // 78.54...
console.log(math.PI);                  // 3.14159...

// Destructure for cleaner syntax
const { add, multiply, divide } = require('./mathUtils');
console.log(add(10, 20));               // 30
console.log(multiply(4, 5));            // 20

// Module paths:
// './mathUtils' → same directory
// '../lib/math' → parent directory
// 'express' → node_modules (npm package)
```

```
greet() { return `Hello, I'm ${this.name}!`; }
toJSON() { return { id: this.id, name: this.name, email: this.email }; }
}
module.exports = User; // export the class itself

// app.js
const User = require('./user');
const alice = new User('Alice', 'alice@example.com');
console.log(alice.greet()); // Hello, I'm Alice!
console.log(alice.toJSON()); // { id: ..., name: 'Alice', ... }
```



Build a temperature converter module and use it in app.js

Task

```
// Task:
// 1. Create tempConverter.js with these functions:
//   - celsiusToFahrenheit(c) → c * 9/5 + 32
//   - fahrenheitToCelsius(f) → (f - 32) * 5/9
//   - celsiusToKelvin(c) → c + 273.15
//   - kelvinToCelsius(k) → k - 273.15
// 2. Export all four functions
// 3. Create app.js that:
//   - requires tempConverter
//   - converts 100°C to Fahrenheit and Kelvin
//   - converts 212°F to Celsius
//   - prints all results
// Expected output:
// 100°C = 212.00°F
// 100°C = 373.15K
// 212°F = 100.00°C
```

✓ Solution

06

File System (fs)

Read, write, watch files and directories

The fs module provides both synchronous (Sync) and asynchronous methods. Prefer async in servers.

Read & Write

```
const fs = require('fs');
const path = require('path');

// — READING FILES —————
// Synchronous (blocks) — use only in startup scripts
const data = fs.readFileSync('data.txt', 'utf8');
console.log(data);

// Asynchronous (preferred in servers)
fs.readFile('data.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('Error:', err.message);
    return;
  }
  console.log(data);
});

// — WRITING FILES —————
// Write (creates or overwrites)
fs.writeFile('output.txt', 'Hello Node!', 'utf8', err => {
  if (err) throw err;
  console.log('File written!');
});

// Append to existing file
fs.appendFile('log.txt', `[${new Date().toISOString()}] Event\n`,
err => {
  if (err) throw err;
});
```

Directories & File Operations

```
// — DIRECTORIES —————
// Create directory
fs.mkdir('logs', { recursive: true }, err => {
  if (err) throw err;
  console.log('Directory created');
});

// List directory contents
fs.readdir('./', (err, files) => {
  if (err) throw err;
  files.forEach(file => {
    const stat = fs.statSync(file);
    console.log(`${stat.isDirectory() ? 'DIR' : 'FILE'} ${file}`);
  });
});

// — FILE INFO —————
fs.stat('data.txt', (err, stats) => {
  console.log('Size:', stats.size, 'bytes');
  console.log('Modified:', stats.mtime);
  console.log('Is file:', stats.isFile());
});

// — DELETE —————
fs.unlink('temp.txt', err => {
  if (err && err.code !== 'ENOENT') throw err;
  console.log('Deleted (if existed)');
});

// — RENAME / MOVE —————
fs.rename('old.txt', 'new.txt', err => {
  if (err) throw err;
  console.log('Renamed!');
});
```

Streams process data in chunks — never load the entire file into memory at once.

Read Stream & Write Stream

```
const fs = require('fs');

// — READ STREAM —————
// Process a large file without loading it all in memory
const readStream = fs.createReadStream('large-file.txt', {
  encoding: 'utf8',
  highWaterMark: 64 * 1024, // chunk size: 64KB
});

let totalChunks = 0;
readStream.on('data', chunk => {
  totalChunks++;
  console.log(`Chunk ${totalChunks}: ${chunk.length} chars`);
});
readStream.on('end', () => {
  console.log(`Done! ${totalChunks} chunks processed.`);
});
readStream.on('error', err => {
  console.error('Read error:', err.message);
});

// — WRITE STREAM —————
const writeStream = fs.createWriteStream('output.txt');

writeStream.write('First line\n');
writeStream.write('Second line\n');
writeStream.end('Final line\n'); // signals end

writeStream.on('finish', () => console.log('Write complete!'));
```

Pipe — Efficient Chaining

```
// — PIPE — chain streams together —————
const fs = require('fs');
const zlib = require('zlib');

// Read → Compress → Write (without loading entire file)
const src = fs.createReadStream('input.txt');
const gzip = zlib.createGzip();
const dest = fs.createWriteStream('input.txt.gz');

src.pipe(gzip).pipe(dest);
dest.on('finish', () => console.log('File compressed!'));

// — SERVE A FILE VIA HTTP STREAM —————
const http = require('http');
const path = require('path');

http.createServer((req, res) => {
  const filePath = path.join(__dirname, 'public', 'index.html');
  res.writeHead(200, { 'Content-Type': 'text/html' });

  // Stream the file directly to the HTTP response
  const stream = fs.createReadStream(filePath);
  stream.pipe(res); // automatic backpressure!

  stream.on('error', err => {
    res.writeHead(404);
    res.end('File not found');
  });
}).listen(3000);
```

07

HTTP Server

Building web servers from scratch with the http module

The built-in http module lets you create a web server without any external packages.

Minimal HTTP Server

```
// server.js - Minimal HTTP server
const http = require('http');

const server = http.createServer((req, res) => {
  // req: incoming request (url, method, headers, body...)
  // res: outgoing response (statusCode, headers, body)

  // Set response headers
  res.writeHead(200, {
    'Content-Type': 'text/plain',
    'X-Powered-By': 'Node.js'
  });

  // Send response body
  res.end('Hello, World!\n');
});

const PORT = 3000;
server.listen(PORT, '127.0.0.1', () => {
  console.log(`Server running at http://127.0.0.1:${PORT}/`);
});

// Run: node server.js
// Visit: http://localhost:3000
```

What Each Part Does

```
// Step-by-step:
// 1. http.createServer(callback) - creates the server
//    callback receives (req, res) for every request

// 2. res.writeHead(statusCode, headers)
//    Common status codes:
//    200 = OK
//    201 = Created
//    400 = Bad Request
//    401 = Unauthorized
//    403 = Forbidden
//    404 = Not Found
//    500 = Internal Server Error

// 3. res.end(body)
//    Sends the response body and closes the connection
//    OR: res.write(chunk) + res.end()

// 4. server.listen(port, host, callback)
//    Starts listening on the given port
```



How to test: Run node server.js in terminal → open browser at <http://localhost:3000> OR use: `curl http://localhost:3000`

Set the Content-Type header correctly so the browser/client knows how to handle the response.

```
Serving HTML // Serve HTML from a file
const http = require('http');
const fs = require('fs');
const path = require('path');

http.createServer((req, res) => {
  const filePath = path.join(__dirname, 'index.html');

  fs.readFile(filePath, 'utf8', (err, html) => {
    if (err) {
      res.writeHead(500);
      return res.end('Server Error');
    }
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end(html);
  });
}).listen(3000);
```

```
Serving JSON // Serve JSON data (REST API style)
const http = require('http');

const users = [
  { id: 1, name: 'Alice', role: 'admin' },
  { id: 2, name: 'Bob', role: 'user' },
];

http.createServer((req, res) => {
  res.writeHead(200, {
    'Content-Type': 'application/json',
    'Access-Control-Allow-Origin': '*', // allow CORS
  });
  res.end(JSON.stringify({ success: true, data: users }));
}).listen(3000);
```

Basic Routing with http module

```
// Basic routing with the http module
const http = require('http');
const fs = require('fs');
const path = require('path');

const users = [{ id: 1, name: 'Alice' }, { id: 2, name: 'Bob' }];

http.createServer((req, res) => {
  const url = req.url;
  const method = req.method;

  if (url === '/' && method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    return res.end('<h1>Welcome to Node.js!</h1>');
  }

  if (url === '/api/users' && method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    return res.end(JSON.stringify(users));
  }

  // 404 fallback
  res.writeHead(404, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ error: 'Not Found' }));
}).listen(3000, () => console.log('Server on :3000'));
```

08

Async Patterns

Callbacks → Promises → async/await

Node.js is fundamentally async. Three patterns evolved over time — `async/await` is current best practice.

Gen 1: Callbacks — "Callback Hell"

```
// GENERATION 1: Callbacks (Node.js original style)
const fs = require('fs');

// Read file → parse JSON → read another file
fs.readFile('users.json', 'utf8', (err, data) => {
  if (err) return console.error(err);
  const users = JSON.parse(data);

  fs.readFile('posts.json', 'utf8', (err2, data2) => {
    if (err2) return console.error(err2);
    const posts = JSON.parse(data2);

    fs.readFile('comments.json', 'utf8', (err3, data3) => {
      if (err3) return console.error(err3);
      const comments = JSON.parse(data3);
      // Finally have all data! But this is "callback hell"
      console.log(users, posts, comments);
    });
  });
});
```

Gen 2: Promises — flat chain

```
// GENERATION 2: Promises (ES6)
const fs = require('fs').promises; // fs.promises — returns Promises

fs.readFile('users.json', 'utf8')
  .then(data => {
    const users = JSON.parse(data);
    return fs.readFile('posts.json', 'utf8'); // return next promise
  })
  .then(data => {
    const posts = JSON.parse(data);
    return fs.readFile('comments.json', 'utf8');
  })
  .then(data => {
    console.log(JSON.parse(data));
  })
  .catch(err => {
    console.error('Any error caught here:', err); // one catch!
  });
```

Gen 3: `async/await` + `Promise.all` — parallel execution

```
// GENERATION 3: async/await (ES2017) — modern best practice
const fs = require('fs').promises;

async function loadAllData() {
  try {
    const [usersRaw, postsRaw, commentsRaw] = await Promise.all([
      fs.readFile('users.json', 'utf8'), // run all three
      fs.readFile('posts.json', 'utf8'), // in parallel!
      fs.readFile('comments.json', 'utf8'),
    ]);
  }
}
```

```
const users = JSON.parse(usersRaw);
const posts = JSON.parse(postsRaw);
const comments = JSON.parse(commentsRaw);
console.log({ users, posts, comments });
} catch (err) {
  console.error('Error:', err.message); // clean error handling
}
loadAllData(); // reads all 3 files simultaneously!
```

 Write an async function that reads a JSON file and filters/transforms the data

Task

```
// students.json (given file):
[

  {"id":1,"name":"Alice","score":88,"grade":"A"},
  {"id":2,"name":"Bob",
   "score":72,"grade":"B"},

  {"id":3,"name":"Charlie","score":55,"grade":"C"}
],

{"id":4,"name":"Dana","score":91,"grade":"A"},
  {"id":5,"name":"Eve", "score":65,"grade":"B"}
]

// Task: Create report.js that:
// 1. Reads students.json using async/await
// 2. Calculates: count, average, highest,
lowest score
// 3. Filters students with grade 'A'
// 4. Writes a summary to report.txt
// 5. Handles errors with try/catch

// Expected report.txt:
// === Student Report ===
// Total: 5 students
// Average score: 74.2
// Highest: Dana (91)
// Lowest: Charlie (55)
// Grade A students: Alice, Dana
```

✓ Solution

09

Express.js

The most popular Node.js web framework

Express adds routing, middleware, and utilities on top of Node's http module.

Without Express

Raw http module

Manual URL parsing

Manual header management

No middleware support

200+ lines for a simple API

No built-in body parsing

Quick Setup

```
# Setup — install Express
npm init -y
npm install express

# Install nodemon for auto-reload during development
npm install nodemon --save-dev

# Add to package.json scripts:
# "dev": "nodemon index.js"

# Then run:
npm run dev
```

With Express

Express.js

router.get('/users', handler)

Sensible defaults + easy overrides

app.use() middleware pipeline

10 lines for the same API

express.json() middleware

First Express App

```
// index.js — Minimal Express app
const express = require('express');
const app = express();
const PORT = 3000;

// Built-in middleware — parse JSON request bodies
app.use(express.json());

// Basic route
app.get('/', (req, res) => {
  res.send('<h1>Hello from Express!</h1>');
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Express makes routing clean and readable. Each route = HTTP method + URL pattern + handler.

All HTTP Methods

```
const express = require('express');
const app = express();
app.use(express.json());

// — HTTP METHOD ROUTES —————
app.get('/users', (req, res) => {
  // GET /users — return all users
  res.json({ users: [] });
});

app.post('/users', (req, res) => {
  // POST /users — create a user
  const { name, email } = req.body; // from JSON body
  res.status(201).json({ id: Date.now(), name, email });
});

app.put('/users/:id', (req, res) => {
  // PUT /users/42 — replace user 42
  const { id } = req.params;
  res.json({ id, ...req.body });
});

app.patch('/users/:id', (req, res) => {
  // PATCH /users/42 — update fields of user 42
  res.json({ updated: true, id: req.params.id });
});

app.delete('/users/:id', (req, res) => {
  // DELETE /users/42 — remove user 42
  res.json({ deleted: true, id: req.params.id });
});

app.listen(3000);
```

Params, Query Strings, Request Object

```
// — ROUTE PARAMETERS —————
// :id and :postId are route parameters
app.get('/users/:id/posts/:postId', (req, res) => {
  const { id, postId } = req.params;
  res.json({ userId: id, postId });
});
// GET /users/5/posts/12 → { userId: '5', postId: '12' }

// — QUERY STRINGS —————
app.get('/search', (req, res) => {
  const { q, page = 1, limit = 10 } = req.query;
  // GET /search?q=node&page=2&limit=5
  res.json({ query: q, page, limit });
});

// — REQUEST OBJECT —————
app.post('/data', (req, res) => {
  console.log(req.method); // 'POST'
  console.log(req.url); // '/data'
  console.log(req.path); // '/data'
  console.log(req.headers); // { 'content-type': 'application/json', ... }
  console.log(req.body); // parsed JSON body { ... }
  console.log(req.params); // route params { id: '...' }
  console.log(req.query); // query string { page: '1' }
  res.json({ received: req.body });
});
```

Express makes routing clean and readable. Each route = HTTP method + URL pattern + handler.

All HTTP Methods

```
const express = require('express');
const app = express();
app.use(express.json());

// — HTTP METHOD ROUTES —
app.get('/users', (req, res) => {
  // GET /users — return all users
  res.json({ users: [] });
});

app.post('/users', (req, res) => {
  // POST /users — create a user
  const { name, email } = req.body; // from JSON body
  res.status(201).json({ id: Date.now(), name, email });
});

app.put('/users/:id', (req, res) => {
  // PUT /users/42 — replace user 42
  const { id } = req.params;
  res.json({ id, ...req.body });
});

app.patch('/users/:id', (req, res) => {
  // PATCH /users/42 — update fields of user 42
  res.json({ updated: true, id: req.params.id });
});

app.delete('/users/:id', (req, res) => {
  // DELETE /users/42 — remove user 42
  res.json({ deleted: true, id: req.params.id });
});

app.listen(3000);
```

Solution

```
const express = require('express');
const app = express();
app.use(express.json());
var users = [];

// — HTTP METHOD ROUTES —
app.get('/users', (req, res) => {
  // GET /users — return all users
  //res.json({ users: [] });
  res.json({ users: users });
});

app.post('/users', (req, res) => {
  // POST /users — create a user
  const { id, name, email } = req.body; // from JSON body
  users.push({ id, name, email });
  res.status(201).json({ id, name, email });
});

app.put('/users/:id', (req, res) => {
  // PUT /users/42 — replace user 42
  const { id } = req.params;
  users = users.map(user => user.id === id ? { id, ...req.body } : user);
  res.json({ updated: true, users });
  //res.json({ id, ...req.body });
});

app.patch('/users/:id', (req, res) => {
  // PATCH /users/42 — update fields of user 42
  const { id } = req.params;
  users = users.map(user => user.id === id ? { ...user, ...req.body } : user);
  res.json({ updated: true, users });
});

app.delete('/users/:id', (req, res) => {
  // DELETE /users/42 — remove user 42
  users = users.filter(user => user.id !== req.params.id);
  res.json({ deleted: true, id: req.params.id });
});

app.listen(3000);
```

Middleware functions have access to `req`, `res`, and `next()` — they run in order for every matching request.

Middleware Examples

```
const express = require('express');
const app = express();

// — GLOBAL MIDDLEWARE (runs for every request) —
// 1. Parse JSON bodies
app.use(express.json());

// 2. Logger middleware
app.use((req, res, next) => {
  const now = new Date().toISOString();
  console.log(`[${now}] ${req.method} ${req.url}`);
  next(); // MUST call next() or request hangs!
});

// 3. CORS middleware
app.use((req, res, next) => {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods',
    'GET, POST, PUT, PATCH, DELETE');
  res.setHeader('Access-Control-Allow-Headers',
    'Content-Type, Authorization');
  next();
});

// — PATH-SPECIFIC MIDDLEWARE —
// Only runs for /admin/* routes
app.use('/admin', (req, res, next) => {
  const token = req.headers['authorization'];
  if (!token || token !== 'Bearer secret123') {
    return res.status(401).json({ error: 'Unauthorized' });
  }
  next(); // authorized — continue
});

app.get('/admin/dashboard', (req, res) => {
  res.json({ data: 'Secret dashboard' });
});

app.listen(3000);
```

Middleware Pipeline Flow

Request comes in

`express.json()` — parse body

Logger — log request

 Auth check — token valid?

 Route Handler — send response

Response sent to client

 Always call `next()` in middleware unless you send a response. Forgetting `next()` leaves the client waiting forever.

10

Building a Mini REST API

Full worked example — CRUD operations with Express

A complete CRUD API for users — no database needed (data stored in-memory array)

users-api.js — Part 1

```
// users-api.js — Complete CRUD REST API
const express = require('express');
const app = express();
app.use(express.json());

// In-memory data store
let users = [
  { id: 1, name: 'Alice', email: 'alice@example.com', role: 'admin' },
  { id: 2, name: 'Bob', email: 'bob@example.com', role: 'user' },
];
let nextId = 3;

// Helper: find user by id
const findUser = id => users.find(u => u.id === parseInt(id));

// GET /users — list all
app.get('/users', (req, res) => {
  res.json({ success: true, count: users.length, data: users });
});

// GET /users/:id — get one
app.get('/users/:id', (req, res) => {
  const user = findUser(req.params.id);
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.json({ success: true, data: user });
});

// POST /users — create
app.post('/users', (req, res) => {
  const { name, email, role = 'user' } = req.body;
  if (!name || !email)
    return res.status(400).json({ error: 'name and email required' });
  const user = { id: nextId++, name, email, role };
  users.push(user);
  res.status(201).json({ success: true, data: user });
});
```

users-api.js — Part 2

```
// PUT /users/:id — full replace
app.put('/users/:id', (req, res) => {
  const idx = users.findIndex(u => u.id === parseInt(req.params.id));
  if (idx === -1) return res.status(404).json({ error: 'Not found' });
  const { name, email, role } = req.body;
  if (!name || !email)
    return res.status(400).json({ error: 'name and email required' });
  users[idx] = { id: users[idx].id, name, email, role: role || 'user' };
  res.json({ success: true, data: users[idx] });
});

// PATCH /users/:id — partial update
app.patch('/users/:id', (req, res) => {
  const user = findUser(req.params.id);
  if (!user) return res.status(404).json({ error: 'Not found' });
  Object.assign(user, req.body); // merge changes
  res.json({ success: true, data: user });
});

// DELETE /users/:id
app.delete('/users/:id', (req, res) => {
  const idx = users.findIndex(u => u.id === parseInt(req.params.id));
  if (idx === -1) return res.status(404).json({ error: 'Not found' });
  const deleted = users.splice(idx, 1)[0];
  res.json({ success: true, deleted });
});

// 404 handler — must be LAST
app.use((req, res) => {
  res.status(404).json({ error: `Route ${req.url} not found` });
});

// Global error handler
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Internal Server Error' });
});

app.listen(3000, () => console.log('API running on :3000'));
```

Use curl in the terminal to test every endpoint of your API.

curl Test Commands

```
# Start the server first:
node users-api.js

# — GET all users —————
curl http://localhost:3000/users
# → {"success":true,"count":2,"data":[...]}

# — GET one user —————
curl http://localhost:3000/users/1
# → {"success":true,"data":{"id":1,"name":"Alice",...}}

# — POST — create new user —————
curl -X POST http://localhost:3000/users \
  -H "Content-Type: application/json" \
  -d '{"name":"Charlie","email":"charlie@test.com"}'
# → {"success":true,"data":{"id":3,"name":"Charlie",...}}

# — PATCH — update one field —————
curl -X PATCH http://localhost:3000/users/1 \
  -H "Content-Type: application/json" \
  -d '{"role":"superadmin"}'

# — DELETE —————
curl -X DELETE http://localhost:3000/users/2
# → {"success":true,"deleted":{"id":2,"name":"Bob",...}}

# — Test 404 —————
curl http://localhost:3000/users/999
# → {"error":"User not found"}
```

API Route Summary

GET	/users	200
GET	/users/:id	200 / 404
POST	/users	201 / 400
PUT	/users/:id	200 / 400 / 404
PATCH	/users/:id	200 / 404
DELETE	/users/:id	200 / 404



Create a full CRUD REST API for a product catalog using Express.js

Task Requirements

```
// Build products-api.js with:
//
// Data structure for each product:
// { id, name, price, category, stock, createdAt }
//
// Endpoints to implement:
// GET    /products           → all products
// GET    /products/:id      → one product
// GET    /products?category=X → filter by category
// POST   /products          → create (require name,
price, category)
// PATCH  /products/:id      → update any field(s)
// DELETE /products/:id      → remove product
//
// Validation rules:
// - name: required, string
// - price: required, must be > 0
// - category: required, string
// - stock: optional, default to 0, must be >= 0
//
// Error responses:
// 400 → validation error (with message explaining what's
wrong)
// 404 → product not found
// 201 → created
// 200 → success
```



Solution (key parts)

These are available everywhere in Node.js without requiring anything.

Globals: `__filename`, `__dirname`, `process`, `timers`

```
// — __filename and __dirname —————
console.log(__filename);
// /home/user/project/app.js (full path to this file)

console.log(__dirname);
// /home/user/project (directory of this file)

// — process —————
console.log(process.env.NODE_ENV); // 'development' or
'production'
console.log(process.env.PORT);      // from .env or shell
console.log(process.argv);          // command-line arguments
// ['node', 'app.js', '--port', '3000']
console.log(process.pid);           // process ID
console.log(process.version);       // Node.js version 'v20.x.x'
process.exit(0);                   // exit (0=success, 1=error)

// — Timers —————
const t = setTimeout(() => {
  console.log('Runs once after 2 seconds');
}, 2000);
clearTimeout(t); // cancel before it fires

const interval = setInterval(() => {
  console.log('Runs every 1 second');
}, 1000);
setTimeout(() => clearInterval(interval), 5000); // stop after 5s

setImmediate(() => {
  console.log('Runs at end of current event loop iteration');
});
```

Environment Variables, `console`, `Buffer`

```
// — Environment Variables (best practice) ———
// .env file (never commit to git!):
// PORT=3000
// NODE_ENV=development
// DB_URL=mongodb://localhost/mydb

// Load .env using dotenv package:
// npm install dotenv
require('dotenv').config();

const PORT = process.env.PORT || 3000;
const ENV = process.env.NODE_ENV || 'development';
console.log(`Running on port ${PORT} in ${ENV} mode`);

// — console —————
console.log('info message');
console.error('error message'); // goes to stderr
console.warn('warning');
console.time('label');          // start timer
// ... some code ...
console.timeEnd('label');       // print elapsed time
console.table([                // print as table
  { name: 'Alice', score: 90 },
  { name: 'Bob',   score: 85 },
]);

// — Buffer (binary data) —————
const buf = Buffer.from('Hello, Node!', 'utf8');
console.log(buf);              // <Buffer 48 65 6c...>
console.log(buf.toString('utf8')); // Hello, Node!
console.log(buf.toString('base64')); // SGVsbG8sIE5vZGUh
```

Key patterns and snippets to keep handy

require module

```
const fs = require('fs');
```

Route param

```
app.get('/u/:id', (req, res) =>
  req.params.id);
```

module.exports

```
module.exports = { fn1, fn2 };
```

Query string

```
const { page = 1 } = req.query;
```

async file read

```
const data = await
  fs.promises.readFile('f.txt', 'utf8');
```

Middleware

```
app.use((req, res, next) => { /* ... */
  next(); });
```

path join

```
path.join(__dirname, 'data', 'file.json')
```

404 handler

```
app.use((req, res) =>
  res.status(404).json({error:'Not found'}));
```

Express app

```
const app = express();
app.use(express.json());
```

Environment var

```
const PORT = process.env.PORT || 3000;
```

GET route

```
app.get('/path', (req, res) =>
  res.json(data));
```

app.listen

```
app.listen(PORT, () => console.log(`Running on
:${PORT}`));
```

You can build web servers with Node.js! 🎉

What we covered today:

- ✓ What is Node.js — V8 engine, event loop, architecture
- ✓ Blocking vs Non-Blocking I/O — why async matters
- ✓ npm — package management and package.json
- ✓ Core Modules — fs, path, os, http, url
- ✓ Custom Modules — module.exports and require()
- ✓ HTTP Server — serving HTML, JSON, and routing
- ✓ Express.js — routing, middleware, REST API design
- ✓ Async Patterns — callbacks → Promises → async/await
- ✓ Building a CRUD REST API with Express

Next Lecture: Vue.js Fundamentals